

Bản trình chiếu: Nhúng phẳng

Gu Nan

2007

Nguồn gốc: Nhúng phẳng

IOI China candidate team papers / 2007 / day 2 / Gu Nan.ppt

1 Mục tiêu và khái niệm

Slide lấy “Honeycomb Corn” trong trại đông 2005 làm ví dụ mở đầu: có những bài toán đồ thị cần biết liệu các cạnh có thể được vẽ không giao nhau hay không. Tính chất ấy gọi là tính phẳng của đồ thị. Khi ta không chỉ cần câu trả lời có/không mà còn cần cấu trúc sau khi vẽ phẳng, thuật toán nhúng phẳng là công cụ phù hợp.

Một đồ thị **vẽ phẳng** được nếu có thể vẽ lại sao cho hai cạnh chỉ gặp nhau tại đỉnh chung. Một đồ thị vẽ phẳng được gọi là **đồ thị phẳng**. **Nhúng phẳng** là cách lưu vẽ phẳng bằng dữ liệu tổ hợp: với mỗi đỉnh, danh sách kề được lưu theo thứ tự quanh đỉnh, thường nói là theo chiều kim đồng hồ. Trong bài nói, “mặt phẳng” được hiểu theo nghĩa tổ hợp này, không phải tọa độ hình học cụ thể.

Với đồ thị G có n đỉnh và m cạnh, mục tiêu của thuật toán là dùng thời gian tuyến tính để phán đoán tính phẳng, và nếu phẳng thì dựng một nhúng phẳng cũng trong $\mathcal{O}(n)$.

2 Kiến thức chuẩn bị

Các công cụ nền là DFS, thành phần song liên thông, đỉnh khớp, sắp xếp đếm và các giá trị lowpoint. Một điều kiện cần nhanh là đồ thị phẳng có không quá $3n - 5$ cạnh trong cách đếm của bài; vượt ngưỡng này thì có thể kết luận không phẳng ngay.

Định lý Kuratowski cho hai trường hợp cổ điển: K_5 và $K_{3,3}$. Một đồ thị không phẳng khi và chỉ khi chứa một đồ thị con đồng phôi với một trong hai đồ thị ấy. Các slide dùng định lý này trong phần chứng minh đúng: khi thao tác nhúng thất bại, ta chỉ ra được một cấu trúc tương ứng với K_5 hoặc $K_{3,3}$.

3 Tổng quan thuật toán thêm cạnh

Thuật toán tạo một đồ thị trung gian G_P , ban đầu rỗng, rồi đưa cạnh của đồ thị gốc vào G_P theo một thứ tự kiểm soát được. Việc đưa một cạnh vào G_P gọi là **nhúng cạnh**.

Để trình bày đơn giản, trước hết xét một thành phần song liên thông. Nếu đồ thị có đỉnh khớp, ta tách tại đỉnh khớp, nhúng từng phần rồi ghép lại. Thực hiện một DFS để có cây DFS và thứ tự DFI. Cạnh nối một đỉnh với con của nó là **cạnh cây**; cạnh nối với hậu duệ không phải con trực tiếp là **cạnh hồi**.

Các đỉnh được xử lý theo thứ tự DFI đảo ngược. Khi xử lý v , trước hết nhúng các cạnh cây đi xuống con của v , sau đó nhúng các cạnh hồi (v, w) . Cạnh nối v với tổ tiên không xử lý ở bước này. Nếu một

cạnh hồi không thể nhúng mà vẫn giữ tính phẳng của G_P , thuật toán kết luận đồ thị không phẳng.

4 Nhúng cạnh và khối

Thao tác nhúng cạnh quan trọng nhất xuất hiện khi thành phần hiện tại có đỉnh khớp r . Ta có thể tách thành phần tại r thành hai phần, gán cho mỗi phần một bản sao của r , nhúng cạnh mới, rồi hợp nhất hai phần lại. Trong quá trình này một phía có thể phải bị đảo để các đỉnh cần thiết vẫn nằm trên mặt ngoài.

Vì vậy G_P không được lưu như một đồ thị phẳng nguyên khối; nó được quản lý bằng các khối song liên thông. Khi thêm cạnh hồi, một số khối có thể được ghép lại. Trong cài đặt, đây không chỉ là cách giải thích mà là cấu trúc cần được duy trì thật sự.

Với mỗi khối, đỉnh có DFI nhỏ nhất được gọi là đỉnh gốc của khối. Vì khối chỉ bị tách khi nhúng cạnh cây, gốc của khối có đúng một con trong khối; cạnh từ gốc đến con đó là **cạnh gốc**. Các bản sao của một đỉnh khớp được gọi là điểm sao chép, còn điểm không phải gốc là điểm gốc ban đầu.

5 Hoạt động ngoài, liên quan và hoạt động trong

Tính chất mà phép đảo phải bảo vệ là: trong suốt quá trình nhúng, mọi đỉnh **hoạt động ngoài** đều phải nằm trên mặt ngoài. Mặt ngoài là chu trình biên tiếp xúc với vùng vô hạn bên ngoài. Nếu một khối chỉ có một cạnh, nó được xử lý như một chu trình suy biến để vẫn có hai hướng đi trên mặt ngoài.

Khi đang xử lý đỉnh v , một đỉnh đã xử lý được gọi là hoạt động ngoài nếu nó có cạnh trực tiếp tới một tổ tiên của v , hoặc trong khối con của nó có đỉnh hoạt động ngoài. Để nhận biết nhanh, mỗi đỉnh lưu:

- **ecpoint**: độ sâu của tổ tiên sớm nhất có thể đi tới bằng một cạnh trực tiếp;
- **lowpoint**: độ sâu tổ tiên sớm nhất có thể đi tới trực tiếp hoặc qua hậu duệ;
- **SDlist**: danh sách lowpoint của các con, đã sắp theo thứ tự tăng.

Vì lowpoint nằm trong miền $1..n$, có thể dùng sắp xếp đếm để xây dựng toàn bộ SDlist trong $\mathcal{O}(n)$. Khi một con được ghép vào cha, xóa mục tương ứng khỏi SDlist; nhờ đó kiểm tra hoạt động ngoài chỉ còn hằng số.

Đỉnh liên quan là đỉnh đã xử lý có cạnh trực tiếp tới v , hoặc có khối con chứa đỉnh liên quan. Đỉnh vừa liên quan vừa không hoạt động ngoài gọi là **hoạt động trong**. Khối cũng được phân loại tương tự: khối hoạt động ngoài, khối liên quan, khối hoạt động trong và khối không hoạt động.

6 Phép đảo

Phép đảo được dùng để đặt các đỉnh hoạt động ngoài về mặt ngoài trước khi nhúng cạnh. Cách trực tiếp là đảo mọi danh sách kề của khối, nhưng quá chậm. Slide dùng một đánh dấu trên cạnh gốc: mọi cạnh cây ban đầu có dấu $+1$; khi cần đảo một khối, đổi dấu cạnh gốc của khối thành -1 . Sau cùng, một DFS trên cây cho biết mỗi đỉnh có bị đảo hay không bằng cách đếm số dấu -1 trên đường từ gốc đến đỉnh.

Trong lúc chạy thuật toán, ta chỉ cần duy trì hai láng giềng của mỗi đỉnh trên mặt ngoài. Khi đảo một khối, không cần biết tuyệt đối chiều kim đồng hồ; chỉ cần biết từ gốc đi theo hướng nào và khi đến một đỉnh từ một phía thì phải rời qua phía còn lại. Vì thế ảnh hưởng của phép đảo lên mặt ngoài có thể được cập nhật bằng vài hoán đổi con trỏ.

7 Hai hàm walkup và walkdown

Khi xử lý cạnh hồi (v, w) , hàm walkup đi từ w dọc mặt ngoài lên phía v để đánh dấu thông tin liên quan. Mỗi đỉnh có một danh sách proots chứa các khối con liên quan. Khi đi từ gốc sao chép của một khối lên điểm gốc ban đầu, ta đưa khối ấy vào proots.

Đường trên mặt ngoài có hai hướng. Để không đi nhầm phía quá dài, walkup đi đồng thời theo cả hai hướng; như vậy chi phí chỉ hơn hằng số lần đường ngắn hơn. Để tránh lặp lại phần chung giữa nhiều cạnh hồi của cùng v , mỗi đỉnh có dấu visited; trong lượt xử lý v , nếu gặp đỉnh đã mang dấu v , quá trình đi lên có thể dừng.

Hàm walkdown dùng thông tin liên quan để thật sự nhúng cạnh hồi. Nó cũng đi trên mặt ngoài, nhưng về nguyên tắc không được đi xuyên qua đỉnh hoạt động ngoài, trừ trường hợp đỉnh ấy còn có khối con liên quan cần đi xuống. Khi gặp một đỉnh liên quan, trước hết nhúng cạnh hồi. Nếu đỉnh không hoạt động ngoài thì tiếp tục đi. Nếu đỉnh hoạt động ngoài nhưng không còn liên quan, đó là đỉnh dừng và lượt đi kết thúc.

Hai quy tắc điều khiển thứ tự đi xuống là:

1. nếu một đỉnh có nhiều khối con liên quan, đi xuống các khối hoạt động trong trước, vì chúng không chứa đỉnh dừng và chắc chắn quay lại được;
2. khi đi xuống một khối liên quan hoạt động ngoài, ưu tiên hướng trực tiếp tới đỉnh hoạt động trong, rồi tới đỉnh vừa liên quan vừa hoạt động ngoài.

Khi phải ghép các khối, thuật toán lưu trên stack các thông tin về điểm gốc, điểm sao chép, hướng đi xuống và hướng quay về. Nếu hai hướng mâu thuẫn, cần đảo khối trước khi ghép.

8 Cạnh ảo và độ phức tạp

Một tối ưu quan trọng là **cạnh ảo**. Nếu walkdown dừng tại đỉnh hoạt động ngoài s , đoạn mặt ngoài vừa đi qua từ một đỉnh trong tới s sẽ không còn hữu ích cho các lần sau. Thuật toán thêm tạm một cạnh (v, s) không có trong đồ thị gốc để rút ngắn mặt ngoài. Mỗi khối gặp nhiều nhất hai đỉnh dừng, nên tổng số cạnh ảo vẫn là $\mathcal{O}(n)$; cuối cùng chỉ cần xóa chúng.

DFS, tính lowpoint, xây SDlist và các thao tác xóa trong SDlist đều tuyến tính. Mỗi lần đảo và mỗi lần nhúng cạnh là hằng số. Trong walkup/walkdown, các đoạn mặt ngoài sau khi được xử lý sẽ bị cạnh thật hoặc cạnh ảo che đi, còn khối hoạt động trong không bị đi lại nhiều lần. Vì đồ thị phẳng có số cạnh tuyến tính và cạnh ảo cũng tuyến tính, tổng thời gian là $\mathcal{O}(n)$.

9 Tính đúng và bài liên quan

Nếu thuật toán thành công, mọi cạnh hồi được nhúng trên mặt ngoài trong khi các đỉnh hoạt động ngoài vẫn được giữ trên mặt ngoài; do đó không tạo giao cắt. Nếu thuật toán thất bại, các slide phân tích các trường hợp: một đỉnh có từ ba khối con liên quan hoạt động ngoài, một khối con cần đi qua hai lần, hai đỉnh hoạt động ngoài liên quan nằm trong cùng vùng, hoặc một đỉnh liên quan bị hai đỉnh dừng chắn. Mỗi trường hợp đều dựng được một đồ thị con đồng phôi với $K_{3,3}$ hoặc K_5 , nên đồ thị gốc thật sự không phẳng.

Các bài được nhắc tới gồm Winter Camp 2004 “Honeycomb Corn”, bài kiểm tra tính phẳng của MIPT và bài “Circuit Board”. Tác giả cũng dẫn tài liệu của John M. Boyer về thuật toán nhúng phẳng tuyến tính bằng thêm cạnh.